__/__/29__

⇒ Heap Sort (Max heap - Bottom up approach)

```
#include <stdio.h>

void heapify(int arr[], int n, int i, int type){
  int extreme = i;
  int left = 2*i + 1;
  int right = 2*i + 2;

  if (type == 1){
    if (left < n && arr[left] > arr[extreme])
      extreme = left;
    if (right < n && arr[right] > arr[extreme])
      extreme = right;
  } else {
    if (left < n && arr[left] < arr[extreme])
      extreme = left;
    if (right < n && arr[right] < arr[extreme])
      extreme = right;
  }
  if (extreme != i){
    int temp = arr[i];
    arr[i] = arr[extreme];
    arr[extreme] = temp;
```

```
            heapify(arr, n, extreme, type);
        }
    }

void buildHeap(int arr[], int n, int type) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i, type);
}

void heapSort(int arr[], int n, int type){
    buildHeap(arr, n, type);

    for (int i = n-1; i > 0; i--){
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0, type);
    }
}

void printArr(int arr[], int n) {
    for(int i=0; i<n; i++)
        printf("%d", arr[i]);
    printf("\n");
}
```

__/__/31__

```
int main(){
    int arr1[] = {4, 10, 3, 5, 1};
    int arr2[] = {4, 10, 3, 5, 1};
    int n = 5;

    heapSort(arr1, n, 1);
    printf("Max Heap (ascending): \n");
    printArray(arr1, n);

    heapSort(arr2, n, 0);
    printf("Min Heap (descending): \n");
    printArray(arr2, n);

    return 0;
}
```

o/p: Max Heap (ascending):

1 3 4 5 10

Min Heap (descending):

10 5 4 3 1

execution time : 0.006 s

Lab.